

Onboarding — Projet de prédiction du trafic cycliste

Documentation officielle

Table des matières

1. Objectif.....	3
2. Prérequis	3
3. Cloner le projet.....	4
4. Structure initiale du repository.....	4
5. Création de l'environnement virtuel.....	5
Windows	5
Linux / macOS	5
6. Étape 1 - Préparation des données	5
7. Étape 2 - Entraînement des modèles	7
8. Étape 3 - Génération des modèles et graphiques.....	7
9. Étape 4 - Génération des Model Cards.....	8
10. Pipeline complet.....	9
Windows	9
Linux / macOS	9
11. Vérification de l'installation.....	10
12. Règles Git.....	11
13. Ajouter un nouveau traitement.....	11
14. Ajouter un nouvel entraînement	12
15. Ajouter un nouveau graphique	13
16. Liens utiles	13

1. Objectif

Ce document explique comment installer et exécuter le projet en local après avoir cloné le dépôt.

Le dépôt Git contient uniquement les données d'entrée et les scripts nécessaires à la reconstruction du projet.

Les éléments suivants sont générés localement et ne sont pas versionnés :

- dataset Silver ;
- modèles entraînés ;
- prédictions ;
- métriques ;
- graphiques ;
- Model Cards.

2. Prérequis

Pour exécuter le projet, il faut disposer de :

- Python 3 ;
- Git ;
- pip ;
- un environnement virtuel Python.

Le projet a été développé avec Python Python 3.13.

3. Cloner le projet

Cloner le dépôt :

```
git clone https://github.com/piTibo/PredictionVelo
```

4. Structure initiale du repository

Après le clone, le projet doit avoir approximativement cette structure :

```
Projet/
|
|— Data/
|   └─ Bronze/
|       └─ eco-counter-data.csv
|
|— Scripts/
|   └─ Bronze2silver.py
|       └─ ModelCreation&ModelCard.py
|
|— .gitignore
|— requirements.txt
└─ ONBOARDING.md
```

Le fichier `eco-counter-data.csv` correspond aux données d'entrée brutes.

Les dossiers et fichiers contenant les données traitées, modèles et graphiques seront créés par les scripts.

5. Création de l'environnement virtuel

Depuis la racine du projet :

Windows

```
python -m venv .env  
.env\Scripts\activate
```

Linux / macOS

```
python3 -m venv .env  
source .env/bin/activate
```

Une fois l'environnement activé, installer les dépendances :

```
pip install -r requirements.txt
```

6. Étape 1 - Préparation des données

Le premier script à exécuter est :

```
` Scripts/Bronze2silver.py `
```

Il permet de transformer les données Bronze en données Silver utilisables pour la modélisation.

Exécution depuis la racine du projet :

```
python Scripts/Bronze2silver.py
```

Le script réalise notamment :

- chargement du CSV Bronze ;
- nettoyage des données ;
- conversion et contrôle des dates ;
- suppression des colonnes inutiles ;
- contrôle des valeurs manquantes ;
- tri chronologique ;
- création des variables temporelles ;
- export du dataset Silver ;
- génération de la documentation associée au traitement.

À la fin de cette étape, le fichier suivant doit notamment être créé :

```
Data/  
└─ Silver/  
    └─ silver_eco-counter-data.csv
```

Script : <https://github.com/piTibo/PredictionVelo/blob/master/Scripts/Bronze2silver.py>

7. Étape 2 - Entraînement des modèles

Une fois le dataset Silver généré, exécuter :

```
python Scripts/ModelCreation&ModelCard.py
```

Sous Windows PowerShell, si nécessaire :

```
python "Scripts/ModelCreation&ModelCard.py"
```

Le script charge les données Silver et entraîne les modèles de prédiction.

L'expérience réalisée :

- XGBoost avec un lag de 168 heures.

Le lag 168h utilise la fréquentation observée une semaine auparavant.

Script :

<https://github.com/piTibo/PredictionVelo/blob/master/Scripts/ModelCreation%26ModelCard.py>

8. Étape 3 - Génération des modèles et graphiques

L'exécution du script de modélisation génère automatiquement les artefacts nécessaires à l'évaluation des modèles.

Pour chaque modèle sont notamment produits :

- modèle entraîné ;
- prédictions ;
- métriques MAE, RMSE et R^2 ;
- graphique de convergence ;
- graphique prédiction / réalité ;
- analyse des résidus ;
- histogramme des résidus ;
- importance des variables ;
- analyse des performances par période de la journée.

Ces fichiers sont générés localement et ne doivent pas être ajoutés au repository Git.

9. Étape 4 - Génération des Model Cards

Les Model Cards sont générées automatiquement par le script d'entraînement.

Elles contiennent notamment :

- description du modèle ;
- données utilisées ;
- variables explicatives ;
- hyperparamètres ;
- performances ;
- comparaison avec une baseline ;
- graphiques ;

- analyse des résidus ;
- limites du modèle.

Il n'est donc pas nécessaire de créer manuellement les Model Cards après l'entraînement.

10. Pipeline complet

Pour reconstruire entièrement le projet après un clone :

Windows

```
git clone <URL_DU_REPOSITORY>
```

```
cd <NOM_DU_PROJET>
```

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
python Scripts/Bronze2silver.py
```

```
python "Scripts/ModelCreation&ModelCard.py"
```

Linux / macOS

```
git clone <URL_DU_REPOSITORY>
```

```
cd <NOM_DU_PROJET>
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install -r requirements.txt
```

```
python Scripts/Bronze2silver.py
```

```
python "Scripts/ModelCreation&ModelCard.py"
```

À la fin de ces commandes, les données Silver, modèles, métriques, graphiques et Model Cards doivent avoir été reconstruits.

11. Vérification de l'installation

Après exécution complète, vérifier que :

- le dataset Silver a été créé ;
- les modèles ont été entraînés et exportés ;
- les fichiers de métriques existent ;
- les prédictions ont été exportées ;
- les graphiques ont été générés ;
- les Model Cards ont été créées.

Si tous ces éléments sont présents, l'environnement local est correctement configuré.

12. Règles Git

Le repository doit contenir :

- les données Bronze nécessaires à l'exécution ;
- les scripts Python ;
- ``requirements.txt`` ;
- `` .gitignore`` ;
- la documentation du projet ;
- ce document d'onboarding.

Le repository ne doit pas contenir :

- l'environnement `` .venv`` ;
- le dataset Silver généré ;
- les modèles entraînés ;
- les prédictions générées ;
- les graphiques générés ;
- les fichiers temporaires Python.

Ces éléments doivent être déclarés dans `` .gitignore`` .

13. Ajouter un nouveau traitement

Pour ajouter un nouveau traitement des données :

1. modifier ou ajouter un script dans `` Scripts/`` ;

2. lire les données depuis `Data/Bronze/` ou `Data/Silver/` selon le besoin ;
3. appliquer le traitement ;
4. exporter le résultat dans le dossier approprié ;
5. documenter le traitement ;
6. vérifier que les fichiers générés ne sont pas ajoutés au repository s'ils sont reproductibles.

Exemple :

```
df["nouvelle_variable"] = ...
```

14. Ajouter un nouvel entraînement

Pour tester un nouveau modèle :

1. charger le dataset Silver ;
2. définir les variables explicatives et la cible ;
3. conserver une séparation temporelle des données ;
4. entraîner le modèle ;
5. calculer au minimum MAE, RMSE et R^2 ;
6. comparer les résultats à une baseline ;
7. exporter le modèle et les métriques ;
8. générer les graphiques nécessaires ;
9. documenter le modèle dans une Model Card.

Les nouveaux modèles doivent être enregistrés dans un dossier distinct afin de ne pas écraser les expériences précédentes.

15. Ajouter un nouveau graphique

Les graphiques sont générés avec Matplotlib.

Exemple :

```
fig, ax = plt.subplots(figsize=(12, 6))
```

```
ax.plot(x, y)
```

```
ax.set_title("Titre du graphique")
```

```
ax.set_xlabel("X")
```

```
ax.set_ylabel("Y")
```

```
fig.tight_layout()
```

```
fig.savefig("<CHEMIN_DU_GRAPHIQUE>", dpi=150)
```

```
plt.close(fig)
```

Le graphique doit être généré automatiquement par le script concerné et non ajouté manuellement au repository.

16. Liens utiles

- Repository : <https://github.com/piTibo/PredictionVelo>

- Script Bronze → Silver :

<https://github.com/piTibo/PredictionVelo/blob/master/Scripts/Bronze2silver.py>

- Script de modélisation :

<https://github.com/piTibo/PredictionVelo/blob/master/Scripts/ModelCreation%26ModelCard.py>

- Documentation technique :

<https://github.com/piTibo/PredictionVelo/blob/master/Documentation%20Technique.pdf>

Documentation officielle